

البنية التحتية للمشروع

مجمع الملك سلمان العالمي للغة العربية — الموقع العام + لوحة التحكم الإدارية

المشروع
المجمع — كرة أرضية + لوحة تحكم

الوثيقة
البنية التحتية

التاريخ
21 مايو 2026

المرحلة
ما قبل الإطلاق

1. مكونات المشروع

المكوّن	الوصف	الجمهور
الموقع العام	صفحة واحدة بكرة أرضية تفاعلية تعرض أنشطة المجمع حول العالم	الزوار العامون
لوحة التحكم	واجهة إدارية لإضافة الأنشطة وتعديلها وحذفها ونشرها	فريق المجمع فقط

2. إدارة المحتوى

- يتحكم فريق المجمع في محتوى الموقع بالكامل عبر لوحة التحكم دون أي تدخل تقني.
- يمكن حفظ النشاط كمسودة ثم نشره لاحقاً — لا يظهر على الموقع العام حتى يُنشر.
- التغييرات المنشورة تنعكس فوراً على الموقع.

3. بيئة التشغيل (Production و Staging)

يُنشر المشروع على بيئتين منفصلتين تماماً لضمان جودة الإصدارات وعدم تأثر المستخدمين بأي تغييرات قبل اعتمادها:

البيئة	النطاق	الغرض	الجمهور
بيئة التجربة (Staging)	staging.example.sa	اختبار التحديثات والمراجعة الداخلية قبل النشر الرسمي — نسخة مطابقة للإنتاج ببيانات اختبارية	فريق المجمع التقني والمحتوى
بيئة الإنتاج (Production)	example.sa	الموقع الفعلي المُتاح للعموم — يستقبل الزيارات الحقيقية	الزوار + فريق المجمع

- كل بيئة معزولة بقاعدة بيانات وتخزين ومتغيرات بيئة (Environment Variables) مستقلة تماماً.
- لا يُنشر أي تغيير على الإنتاج قبل اعتماده على بيئة التجربة.

4. الاستضافة والأمان والإتاحة العالية (High Availability)

صُمّمت بنية الإنتاج لتكون عالية الإتاحة (HA) مبنية على Kubernetes (Azure AKS)، بحيث لا يتسبب تعطل أي مكوّن منفرد في توقّف الموقع:

- يعمل التطبيق داخل حاويات Docker تُدار بواسطة **Kubernetes (Azure AKS)**.
 - **NGINX Ingress Controller**: نقطة الدخول الوحيدة المكشوفة للإنترنت داخل ال cluster؛ يستقبل حركة HTTPS ويوزّعها على pods التطبيق ويُجري فحوصات صحة دورية لاستبعاد أي pod متعطّل تلقائياً.
 - **تعدّد نسخ التطبيق (Replicas)**: نسختان على الأقل من pods التطبيق تعملان في وضع Active-Active داخل K8s Deployment مع استراتيجية RollingUpdate — لا يُزال أي pod قديم إلا بعد أن يصبح الجديد جاهزاً.
 - **فحوصات الصحة (Health Probes)**: كل pod يكشف مسارين لل K8s:
 - `/api/health/live` — Liveness Probe: يتحقق من أن العملية حية؛ يُعيد K8s تشغيل ال pod إذا فشل.
 - `/api/health/ready` — Readiness Probe: يتحقق من اتصال قاعدة البيانات؛ يُوقف K8s توجيه الطلبات إلى ال pod حتى ينجح.
 - **قاعدة البيانات**: Azure Database for PostgreSQL Flexible Server كخدمة مُدارة خارج ال cluster — تشمل نسخاً احتياطية يومية مدمجة وتحويلاً تلقائياً (Automatic Failover) عند التعطّل.
 - **نسخ احتياطي يومي (Backups)**: تُدار بالكامل من Azure — نسخ احتياطية مشفرة تُحفظ 30 يوماً على الأقل.
 - **عزل البيئات**: لكل بيئة (staging/production) namespace Kubernetes مستقل مع secrets و configmap خاصّين بها.
 - **مراقبة (Monitoring)**: مراقبة مستمرة لزمان الاستجابة، استهلاك الموارد، وحالة ال pods مع تنبيهات فورية.
 - لوحة التحكم محمية بمصادقة قوية ومستبعدة من محركات البحث.
- هدف مستوى الخدمة (SLA): إتاحة **99.9%** شهرياً، مع زمن استرجاع (RTO) أقل من 15 دقيقة وزمن فقد بيانات (RPO) أقل من 5 دقائق.

5. متطلبات البنية التحتية السحابية (Azure)

تعتمد البنية على خدمات Azure المُدارة بدلاً من خوادم VPS مستقلة، مما يُقلّل عبء الإدارة ويضمن الإتاحة العالية:

موارد Azure المطلوبة

الخدمة	الغرض	المواصفات المقترحة
Azure Kubernetes Service (AKS)	تشغيل وإدارة حاويات التطبيق	Cluster واحد — node pool للإنتاج: نوع Standard_D2s_v3 (2) node pool لل staging: عقدة واحدة من نفس النوع (vCPU / 8 GB × 2 عقدة على الأقل؛
Azure Container Registry (ACR)	تخزين صور Docker المبنية	Basic tier (يمكن ترقيته ل Standard عند زيادة الحجم)
Azure Database for PostgreSQL — Flexible Server	قاعدة البيانات المُدارة خارج ال cluster	Burstable B2s لل staging؛ General Purpose D2ds_v4 للإنتاج — مع تفعيل High Availability و Auto-backup
Azure DevOps	CI/CD Pipelines لبناء الصور ونشرها تلقائياً	مستودع Git + Pipelines + Variable Groups للأسرار

Kubernetes Namespaces

البيئة	Namespace	الغرض
بيئة التجربة	ksgaal-staging	تشغيل نسخة التجربة بإعدادات وأسرار مستقلة
بيئة الإنتاج	ksgaal-production	تشغيل نسخة الإنتاج بإعدادات وأسرار مستقلة

لا توجد خوادم VPS مستقلة — كل الحسابات تعمل داخل AKS على nodes يُديرها Azure تلقائياً. قاعدة البيانات PostgreSQL خارج ال cluster تماماً كخدمة مُدارة.

1.5. بناء الحاويات (Docker)

يُبنى التطبيق كصورة Docker متعددة المراحل (Multi-Stage Build) لتقليل الحجم النهائي وضمان عدم تسريب أدوات التطوير في بيئة الإنتاج:

المرحلة	الغرض
base	صورة node:20-alpine الأساسية مع libc6-compat
deps	تثبيت حزم npm (npm ci) ونسخ ملفات Prisma لتوليد ال client
builder	استقبال NEXT_PUBLIC_* ك build-args وتشغيل prisma generate ثم npm run build — ينتج standalone output في .next/standalone/
runner	صورة نهائية خفيفة تحتوي فقط على .next/standalone والأصول الثابتة — تعمل بمستخدم غير root (nextjs:1001)

- **Standalone Output:** يُفَعَّل في next.config.ts بـ output: "standalone" — يُنتج server.js منفرداً لا يحتاج node_modules الكاملة عند التشغيل.
- **NEXT_PUBLIC_* build-time baking:** المتغيرات العامة (مثل NEXT_PUBLIC_FRONTEND_URL) تُحقن وقت البناء عبر --build-arg ، لذا تُبنى صورة منفصلة لكل بيئة (staging ≠ production).

6. المنافذ المفتوحة واتجاه حركة البيانات

في بنية Kubernetes، نقطة الدخول الوحيدة للإنترنت هي **NGINX Ingress Controller** الذي يحمل عنوان IP عاماً واحداً. جميع المنافذ الداخلية تبقى داخل ال cluster:

المنفذ	البروتوكول	الاتجاه	الغرض
80	HTTP	وارد — NGINX Ingress فقط	إعادة التوجيه التلقائي إلى HTTPS
443	HTTPS	وارد — NGINX Ingress فقط	حركة المرور العامة؛ TLS يُنهي عند ال Ingress
3000	HTTP	داخلي — داخل ال cluster فقط	منفذ التطبيق (Next.js) — لا يُكشف خارجياً
5432	TCP	صادر من ال cluster إلى Azure PostgreSQL	اتصال التطبيق بقاعدة البيانات الخارجية المُدارة
كل المنافذ	—	صادر	سحب صور Docker من ACR، تحديثات الحزم، اتصالات SSL

7. متطلبات الوصول والإعداد

لإتمام عملية الإعداد والنشر، يحتاج الفريق التقني إلى ما يلي من المجمع:

- اشتراك Azure نشط مع صلاحية إنشاء موارد (Owner أو Contributor).
- مشروع Azure DevOps مع صلاحية إنشاء Pipelines و Variable Groups و Service Connections.
- اسمًا نطاق الإنتاج والتجربة — سيزود الفريق التقني المجمع بعنوان IP ال Ingress Controller بعد اكتمال إعداد ال cluster، ويتولى المجمع توجيه A Record كل نطاق إلى هذا العنوان من لوحة إدارة DNS الخاصة بهم.
- ملفات شهادة SSL لكلا النطاقين (ملف الشهادة، المفتاح الخاص، وسلسلة CA) — سيستلمها الفريق التقني ويثبتها ك Kubernetes Secret على ال Ingress Controller.
- إعدادات SMTP (مضيف، منفذ، بيانات المصادقة) لإرسال رسائل OTP وإعادة تعيين كلمة المرور.

لا يلزم الوصول المباشر إلى خوادم VPS — إدارة AKS تتم من Azure Portal أو Azure CLI أو عبر Azure DevOps Pipelines.

8. النطاقات وشهادات SSL

المهمة	المسؤول	التفاصيل
توفير اسمي النطاق	المجمع	نطاق الإنتاج (مثل <code>example.sa</code>) ونطاق التجربة (مثل <code>staging.example.sa</code>)
ضبط سجلات DNS	المجمع	توجيه A Record لكل نطاق إلى عنوان IP ال NGINX Ingress Controller — يزود الفريق التقني المجمع بعنوان IP ال Ingress بعد اكتمال إعداد ال cluster، ويُجري المجمع التعديل من لوحة DNS الخاصة بهم
توفير شهادات SSL	المجمع	ملف الشهادة (<code>.crt</code>)، المفتاح الخاص (<code>.key</code>)، وسلسلة CA لكلا النطاقين
تثبيت شهادات SSL	الفريق التقني	تحميل الشهادات ك Kubernetes TLS Secret وربطها بإعداد ال Ingress لكل بيئة

9. سير النشر والتسليم (Azure DevOps CI/CD)

النشر مؤتمت بالكامل عبر Azure DevOps Pipelines — لا يتطلب أي تدخل يدوي على الخوادم:

- الكود مخزن في مستودع Git مع فرعين رئيسيين: `main` للإنتاج و `develop` للتجربة.
- دورة النشر:** أي تحديث يُنشر أولاً على بيئة التجربة ← يُراجع ويُعتمد من فريق المجمع ← ثم يُدمج في `main` وينشر على الإنتاج تلقائياً.

خطوات ال Pipeline عند كل نشر

#	الخطوة	التفاصيل
1	بناء صورة Docker	يُشغّل <code>docker build</code> مع تمرير <code>NEXT_PUBLIC_*</code> ك <code>build-args</code> مأخوذة من Variable Group البيئة — تُنتج صورة مستقلة لكل بيئة
2	رفع الصورة إلى ACR	<code>docker push</code> إلى Azure Container Registry بوسم يتضمن رقم البناء
3	إنشاء Kubernetes Secret	يُنشأ أو يُحدّث secret يحتوي <code>DATABASE_URL</code> ، <code>JWT_SECRET</code> ، <code>SMTP</code> ، بيانات الأدمن — القيم تأتي من ADO Secret Variables ولا تُسجّل في أي ملف
4	تطبيق ال overlay	<code>kubectl apply -k overlays/staging/</code> أو <code>overlays/production/</code> — يُنشئ namespace ويُطبّق ConfigMap و Service و Ingress
5	تشغيل ترحيل قاعدة البيانات	يُحذف Job القديم ثم يُطبّق <code>migration-job.yaml</code> الذي يُشغّل <code>prisma migrate deploy</code> — Pipeline يتوقف هنا في حال الفشل ولا ينتقل للنشر
6	انتظار اكتمال الترحيل	<code>kubectl wait --for=condition=complete --timeout=300s job/ksgaal-activity-map-migrate</code>
7	نشر التطبيق	<code>kubectl apply -f base/deployment.yaml</code> — يُطبّق RollingUpdate فلا يُزال pod قديم إلا بعد جاهزية الجديد
8	انتظار اكتمال ال rollout	<code>kubectl rollout status deployment/ksgaal-activity-map --timeout=300s</code> — يفشل ال pipeline إذا لم تكتمل خلال 5 دقائق
9	اختبار دخاني (Smoke test)	التحقق من <code>/api/health/live</code> و <code>/api/health/ready</code> — كلاهما يجب أن يُعيد HTTP 200

جميع الأسرار (كلمات المرور، مفاتيح JWT، بيانات قاعدة البيانات) محفوظة في Azure DevOps Secret Variable
Groups ولا تُخزن في ملفات الكود أو سجلات الـ pipeline.

10. تقنيات المنصة (Application Stack)

يوضح هذا القسم التقنيات المعتمدة في بناء المنصة، شاملاً محرك قاعدة البيانات والـ Application Stack الكاملة.

أولاً: محرك قاعدة البيانات

البند	التفاصيل
نوع قاعدة البيانات	Relational (علائقية)
المحرك (Engine)	PostgreSQL
طريقة الوصول (ORM)	Prisma ORM
بروتوكول الاتصال	DATABASE_URL (postgres://)
الكيانات الرئيسية	Users · Organizations · Countries · Activities · Activity Types · Audit Logs

ثانياً: Application Stack

الطبقة	التقنية	الإصدار
Runtime	Node.js LTS	20 (مُثَبَّت في Dockerfile و package.json engines)
Frontend Framework	Next.js (App Router, Standalone Output)	16.2.6
UI Library	React	19.2.0
Language	TypeScript	x.5
Styling	Tailwind CSS	x.4
ORM	Prisma	6.19.3
قاعدة البيانات	PostgreSQL (Azure Flexible Server)	+16
Validation	Zod	4.1.12
Animation	Framer Motion	x.12
3D Rendering	Three.js	0.184.0
Interactive Maps	MapLibre GL	5.24.0
Authentication	JWT (jose) + bcryptjs	jose 6.2.3
Forms	React Hook Form	7.66.1

ثالثاً: بنية النشر والتشغيل

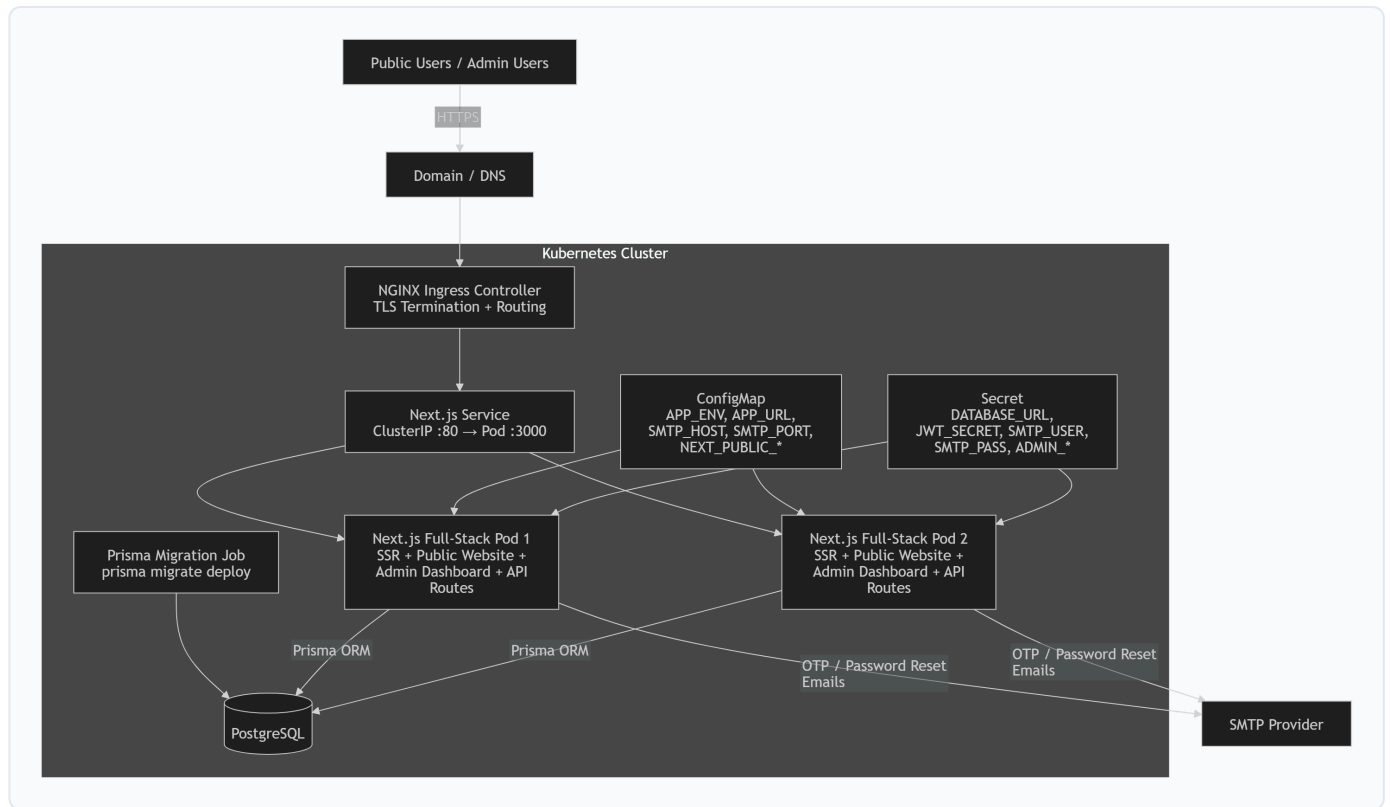
الدور	التقنية	الطبقة
بناء صورة خفيفة بـ 4 مراحل — الصورة النهائية تعمل بـ <code>node server.js</code> فقط	Docker (Multi-Stage Build)	Container Runtime
إدارة الـ pods, Rolling Updates, Health Probes, namespace isolation	Kubernetes (Azure AKS)	Container Orchestration
نقطة الدخول الوحيدة — TLS termination + routing	NGINX Ingress Controller	Ingress Controller
base مشترك + overlays مستقلة للـ staging والإنتاج	Kustomize (Overlays)	Manifest Management
تخزين صور Docker المبنية	Azure Container Registry (ACR)	Container Registry
بناء ورفع ونشر تلقائي عند كل push	Azure DevOps Pipelines	CI/CD
الأسرار محفوظة في ADO ومُمرّرة عند النشر إلى K8s Secret — لا تُخزّن في الكود	ADO Variable Groups + K8s Secrets	Secrets Management
يُشغّل قبل كل نشر — الـ pipeline يتوقف إذا فشل	Kubernetes Job (prisma migrate deploy)	Database Migration

جميع التقنيات المذكورة مفتوحة المصدر باستثناء خدمات Azure. إصدار Node.js مُثبت على LTS 20 في Dockerfile وفي حقل `engines` من `package.json`.

11. المخططات البيانية

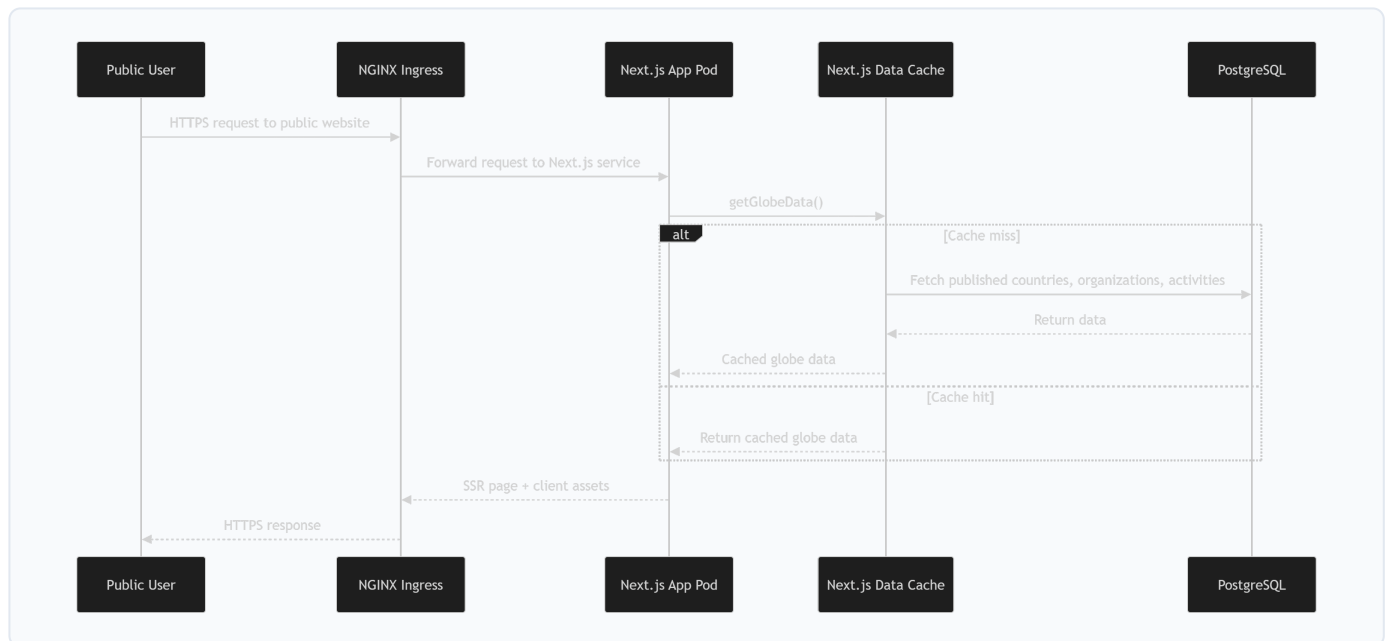
أ. بنية Kubernetes في وقت التشغيل

يوضّح المخطط مكونات الـ cluster الكاملة: المستخدمون ← PostgreSQL + SMTP ← Pods ← Service ← NGINX Ingress ← DNS. يُبيّن أيضاً ConfigMap و Secret و Prisma Migration Job.



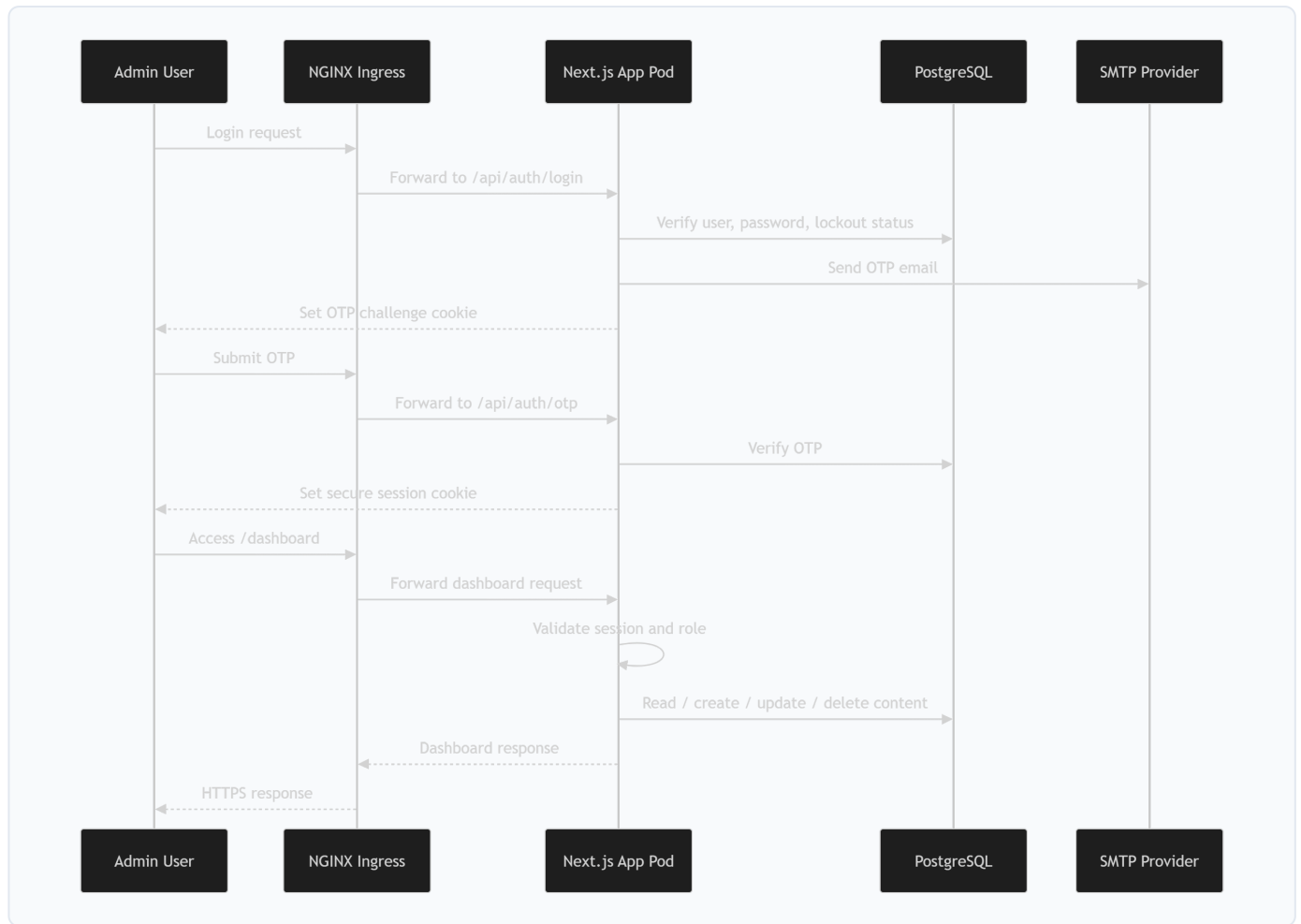
ب. تدفق طلب المستخدم العام (Public Request Flow)

مخطط تسلسلي يُتابع طلب HTTPS من الزائر العام حتى استجابة الصفحة، مع منطق Cache Hit / Cache Miss لبيانات الكرة الأرضية.



ج. تدفق مصادقة المدير ولوحة التحكم (Admin Auth Flow)

مخطط تسلسلي يُوضِّح خطوات تسجيل الدخول: التحقق من كلمة المرور → إرسال OTP عبر SMTP → التحقق من OTP → إنشاء جلسة آمنة → الوصول إلى لوحة التحكم.



د. خط أنابيب النشر التلقائي (Azure DevOps CI/CD Pipeline)

يُبين المخطط الرحلة الكاملة من لحظة دفع الكود حتى وصوله إلى الـ cluster: التحقق من الكود ← بناء صورة Docker ← رفعها إلى ACR ← نشرها على K8s ← تشغيل Rolling Update Migration Job.

